

Mobile Money Fraud Alarm

What "99% accuracy" hides, and an alarm that keeps working when thieves change their habits

Bih Brandy Sango · Data science and engineering · November 2026

A fraud alarm that has to work when thieves adapt

1

The problem

Thieves who get into a wallet send the money to a mule account, then cash it out at an agent. Public models claim about 99% accuracy.

2

What I did

Repeated one of those projects, showed why the number misleads, and built an alarm tested against smarter thieves.

3

What changed

In the hardest test it catches 73% of thefts against 25%, stops 96.5% of stolen money, and is honest that 4 in 10 alerts are honest customers.

The data: a simulated mobile money service

6.36 M
transactions over 31 days

8,213
thefts

0.13%
of transactions are thefts, about 1 in 800

2
types hold all thefts: transfer and cash-out

PaySim was built by researchers from a real African operator's logs (Kaggle, CC BY-SA 4.0). It is simulated: MTN MoMo and Orange Money are the setting for this project, not its source.

Method: four stages

1

Repeat the claim

Balanced data, random split, accuracy: 98.6%.

2

Test honestly

Train on days 1-18, test on days 22-31. Count thefts caught with 300 reviews a day.

3

Find the giveaway

97.8% of thefts send the exact balance; honest customers never do.

4

Make it harder

Thieves take 60-95%, break the money chain and use a funded mule. Then build an alarm that copes.

Decisions, and why I made them

A review budget, not accuracy

With 1 theft in 800, flagging nothing is 99.87% accurate. I count thefts a team catches with 300 calls a day.

Only what's known at the time

Share of balance sent, hour, receiver's history and the money chain (63% of thefts, 0.1% of honest).

Practise on smarter thieves

Extra training copies of thefts that take only part of the balance, so no single shortcut works.

Never show the raw score

The demo says "check with the customer", is only "fairly confident", and adds the 4-in-10 note.

The "99% accuracy" doesn't survive real traffic

98.6%

accuracy on balanced data, random split (the claim)

97.9%

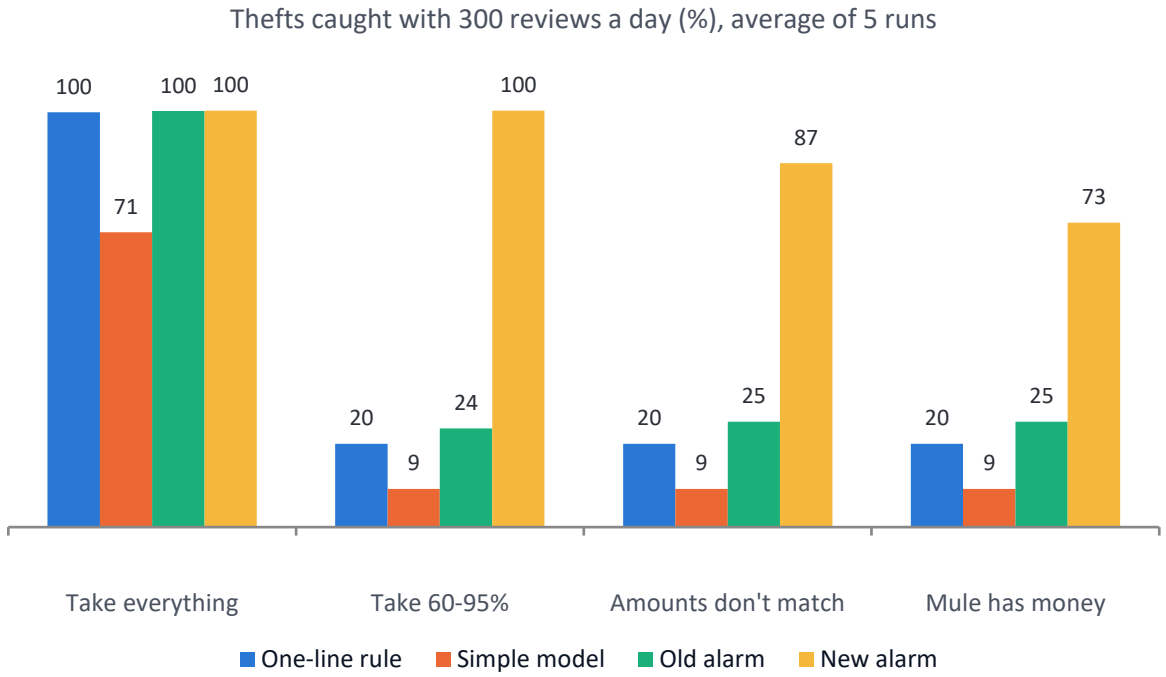
the same model on the last ten days of real traffic

99.1%

accuracy if you call every transaction honest

2,566 of its 5,152 alerts were innocent customers. A high accuracy score hid a model that was worse than doing nothing.

Only the new alarm keeps up when thieves adapt



73%

of thefts caught in the hardest test (65-81% across runs), against 25% for the old alarm

96.5%

of the stolen money stopped, because it catches the large thefts first

False alarms: the score ranks, it doesn't predict

13% → 40%

of alerts are honest customers: as simulated, then in the hardest test (about 121 calls a day)

In one test, scores of 50-80% were thefts only 2.5% of the time. In another, scores of 5-20% hid thefts 64% of the time. So the demo never shows the percentage.



What it can't tell you, and what I'd do next

Limits

- Simulated data: shows how to build and test an alarm, not how it would do inside MTN or Orange
- PaySim has habits real fraud may not: thieves empty the account, mules start at zero
- The test days hold more fraud (0.87%) than real life
- The new alarm practised on thefts close to two of the tests

Next steps

- Retrain on an operator's own confirmed fraud cases
- Recalibrate the score before treating it as a probability
- Keep reporting false alarms next to thefts caught

A 99% headline can hide a model that doesn't work. A good fraud alarm keeps working when thieves adapt, and is honest about who it wrongly flags.

Thank you. Questions?

Project page: bihbrandysango-cloud.github.io/projects/fraud-alarm
Live demo: pixel-perfect-page-615.lovable.app

Backup: questions you may ask

- Isn't 100% on some of your tests too good to be true?
- Would this work at MTN or Orange?
- How do you deal with false positives?
- Why did you choose LightGBM?
- What is SHAP, in simple terms?
- What was the hardest part?
- How do you know the website demo gives the same answers as your model?